



Erhvervsakademi Sydvest
PBA Softwareudvikling (online)

Recipe Recommendation System **– et teknologistudie**

Bachelorprojekt for Professionbachelor i Softwareudvikling

Studerende: Lærke Benedikte Imeland



Projektperiode: 20.10.2025 - 05.01.2026

Vejleder: Patrick Stolc

Antal tegn: 40851

Antal normalsider: 17

Indholdsfortegnelse

Indledning.....	2
Problemstilling	2
Metodevalg.....	2
Kanban	4
Analyse	6
Monolit VS Microservices.....	6
Bounded Contexts	7
Hvordan skal den microservice-baserede arkitektur se ud?	7
Teknologivalg.....	9
Design	11
C4-modellen til softwarearkitektur	11
Implementering af systemet.....	18
Flowet for “Create Recipe”	18
Flowet for “Get Recommendation”	23
Refleksion	31
Konklusion.....	31
Litteraturliste.....	32

Indledning

Dette bachelorprojekt er et teknologistudie uden virksomhed med det formål at undersøge, hvilke teknologier, arkitekturer og designmønstre der er velegnede til at udvikle en skalerbar og robust applikation baseret på microservices.

Undervejs i projektet vil der blive udviklet et proof-of-concept system kaldet Recipe Recommendation System, der kan foreslå opskrifter baseret på ingredienser, brugeren har i sit køleskab, og dermed demonstrerer de valgte teknologier og beslutninger i projektet.

Teknologier vil blive sammenlignet, og der vil blive argumenteret for de bedste teknologier at implementere i systemet, og det vil blive beskrevet, hvordan de implementeres i praksis i proof-of-concept systemet.

Projektet vil inddrage emner fra fagene "System Integration", "Databases for Developers" og "Udvikling af store systemer" og vil blandt andet diskutere disse emner: Valg af arkitektur, kommunikation mellem systemets dele, skalerbarhed, robusthed, datamodellering og databasetype.

Problemstilling

Hvordan kan et stort microservice-baseret system designes med fokus på ikke-funktionelle krav som skalerbarhed og robusthed?

- Hvordan kan forskellige arkitekturprincipper og designmønstre anvendes for at opnå skalerbarhed og robusthed i et microservice-baseret system?
- Hvordan kan et system generere relevante anbefalinger til en stor gruppe slutbrugere ud fra et fælles datasæt uden anvendelse af machine learning?
- Hvorvidt har valg af teknologi indflydelse på et systems skalerbarhed, robusthed og performance?

Metodevalg

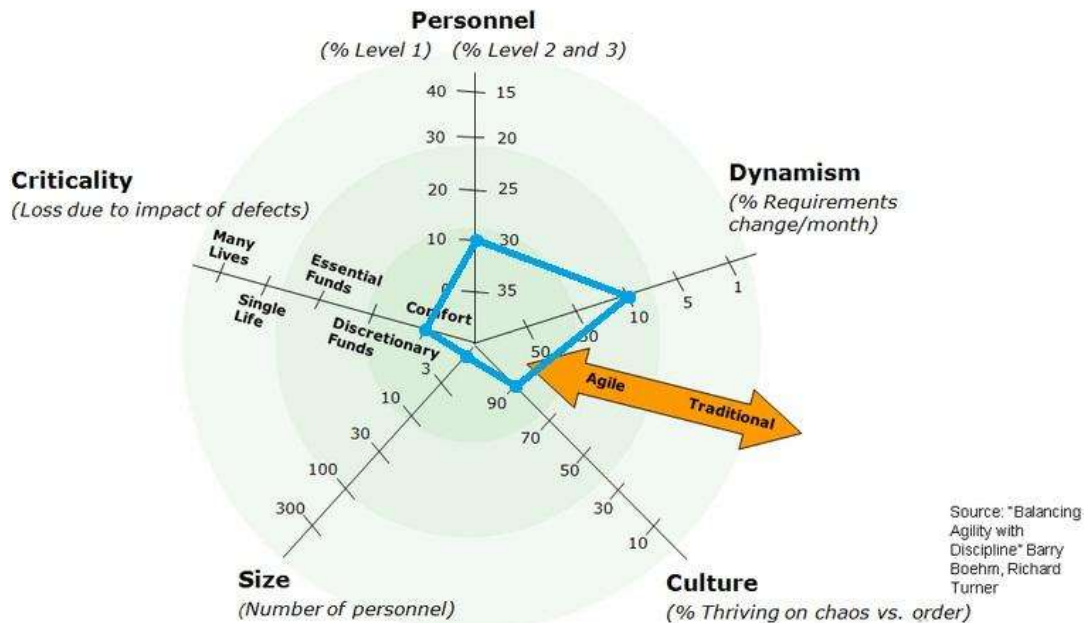
Selvom projektet er uden kunde og uden gruppe, så er det alligevel vigtigt at beslutte, hvilken udviklingsmetode der skal gøres brug af. Valg af udviklingsmetode kommer til at påvirke projektets udformning både ift. planlægning af arbejdsdagene, men også ift. det proof-of-concept system som udgør den praktiske del af projektet.

Man kan enten benytte plandreven eller agil udviklingsmetode og for at identificere, hvilken der er bedst til projektet, kan man udfylde et Boehm og Turner's polar chart.

Boehm og Turner's polar chart har fem akser, som dækker:

- "Personnel": Udviklingsteamets kompetenceniveau. Jo tættere mod midten, jo mere erfaring.
- "Dynamism": Hvor ofte kravene for projektet ændrer sig. Jo tættere mod midten, jo større chance for at kravene ændrer sig.

- “Culture”: Den organisatoriske og arbejdsmæssige kultur i den virksomhed, udviklingsteamet er ansat i. Jo tættere mod midten, jo mere laid-back og ustruktureret er kulturen.
- “Size”: Udviklingsteamets størrelse.
- “Criticality”: Hvor kritiske fejl og systemnedbrud er.[1]



Ovenfor ses det polar chart, som jeg har udfyldt for projektet:

- “Personnel”: Udviklingsteamet består kun af mig selv, og jeg har vurderet mig selv som overvejende erfaren og kompetent ift. de krav, jeg skal opfylde med projektet, men jeg er stadig studerende og i gang med at lære.
- “Dynamism”: Da projektet er et teknologistudie, hvor formålet er at analysere sig frem til den bedste måde at designe dette system på, antages det, at de overordnede design- og teknologivalg, der er truffet, ikke skal laves om. Der er ingen kunde til at ændre i kravene, og det forventes derfor kun at være mig selv evt. i samarbejde med min vejleder, som ændrer i kravene.
- “Culture”: Ingen faste møder eller milepæle, og som ene udvikler er planlægning og struktur selvorganiseret.
- “Size”: Én udvikler.
- “Criticality”: Projektet dækker udviklingen af et opskriftsanbefalingssystem, og skulle der ske systemnedbrud, vil det højst føre til irritation hos brugerne.

Den femkant, der dannes af de placerede punkter på akserne, ligger tæt mod midten, hvilket indikerer, at en agil udviklingsmetode er det bedste valg for projektet.

Jeg har valgt at benytte Kanban for sin enkelthed og overskuelighed, og fordi jeg vurderer, at en agil udviklingsmetode med mange regler og principper som Scrum eller Extreme Programming (XP) vil være at overgøre det og være ikke at udnytte min tid optimalt.

Kanban

Kanban gør brug af et Kanban-board[2] for at skabe overskuelighed i de arbejdsopgaver, der skal laves, og for at kunne udfylde boardet bedst muligt skal jeg først estimere, hvor stort projektets scope skal være, og dermed hvordan jeg skal bruge den tid, jeg har til rådighed.

Til at hjælpe mig selv med at estimere scope, har jeg valgt at bruge en sprint-lignende opdeling af mine arbejdsdage og så give en realistisk vurdering af, hvor meget jeg tænker, jeg kan nå i samarbejde med min vejleder.

Min tidsplan dækker projektperioden fra d. 20-10-2025 til d. 05-01-2026, og den er delt op i følgende tidsblokke.

Tidsblok 1 (20. oktober - 2. november)

<u>To Do</u>	<u>In Progress</u>	<u>Done</u>
• Skriv indledning og konklusion • Skriv metodevalg • Skriv analyseafsnit • Skriv teknologiafsnit • Skriv designafsnit • Skriv implementering • Skriv refleksion • Lav C4 diagrammer • Opret opskrift • Updater opskrift • Slet opskrift • Opret opskriftsanbefaling • Hent opskriftsanbefaling • Opsæt Kafka • Opsæt Neo4j • Opsæt MongoDB • Opsæt Redis	• Skriv problemstilling • Undersøg arkitekturprincipper • Undersøg designmønstre • Undersøg teknologier	

I denne tidsblok var fokus at skrive problemstillingen og undersøge, hvilke arkitekturprincipper, designmønstre og teknologier der ville egne sig bedst til projektet.

Tidsblok 2 (3. november - 16. november)

<u>To Do</u>	<u>In Progress</u>	<u>Done</u>
• Skriv indledning og konklusion • Skriv implementering • Skriv refleksion • Opret opskrift • Updater opskrift • Slet opskrift • Opret opskriftsanbefaling • Hent opskriftsanbefaling • Opsæt Kafka • Opsæt Neo4j • Opsæt MongoDB	• Skriv metodevalg • Skriv analyseafsnit • Skriv teknologiafsnit • Lav C4 diagrammer • Skriv designafsnit	• Skriv problemstilling • Undersøg arkitekturprincipper • Undersøg designmønstre • Undersøg teknologier

• Opsæt Redis		
---	--	--

I denne tidsblok var fokus at skrive rapport om udviklingsmetode samt arkitektur-, design- og teknologibeslutninger truffet i den forrige tidsblok.

Tidsblok 3 (17. november - 30. november)

<u>To Do</u>	<u>In Progress</u>	<u>Done</u>
• Skriv indledning og konklusion • Skriv implementering • Skriv refleksion • Opret opskriftsanbefaling • Hent opskriftsanbefaling • Opsæt Redis	• Opret opskrift • Updater opskrift • Slet opskrift • Opsæt MongoDB • Opsæt Kafka • Opsæt Neo4j	• Skriv problemstilling • Undersøg arkitekturprincipper • Undersøg designmønstre • Undersøg teknologier • Skrive metodevalg • Skriv analyseafsnit • Skriv teknologiafsnit • Lav C4 diagrammer • Skriv designafsnit

I denne tidsblok blev microservicen RecipeManageService implementeret, og MongoDB, Kafka og Neo4j opsat.

Tidsblok 4 (1. december - 14. december)

<u>To Do</u>	<u>In Progress</u>	<u>Done</u>
• Skriv indledning og konklusion • Skriv implementering • Skriv refleksion	• Opret opskriftsanbefaling • Hent opskriftsanbefaling • Opsæt Redis	• Skriv problemstilling • Undersøg arkitekturprincipper • Undersøg designmønstre • Undersøg teknologier • Skrive metodevalg • Skriv analyseafsnit • Skriv teknologiafsnit • Lav C4 diagrammer • Skriv designafsnit • Opret opskrift • Updater opskrift • Slet opskrift • Opsæt MongoDB • Opsæt Kafka • Opsæt Neo4j

I denne tidsblok blev microservicerne RecommendationReadService og RecipeGraphProjectionService implementeret, og Redis opsat.

Tidsblok 5 (15. december - 4. januar)

<u>To Do</u>	<u>In Progress</u>	<u>Done</u>
	• Opret opskriftsanbefaling • Hent opskriftsanbefaling • Opsæt Redis	• Skriv problemstilling • Undersøg

	• Skriv indledning og konklusion • Skriv implementering • Skriv refleksion	arkitekturprincipper • Undersøg designmønstre • Undersøg teknologier • Skriv metodevalg • Skriv analyseafsnit • Skriv teknologiafsnit • Lav C4 diagrammer • Skriv designafsnit • Opret opskrift • Updater opskrift • Slet opskrift • Opsæt MongoDB • Opsæt Kafka • Opsæt Neo4j
--	--	--

I denne tidsblok er microservicerne RecommendationReadService og RecipeGraphProjectionService stadig ved at blive implementeret, da det viste sig at være mere komplekst end først antaget. Derudover bliver de sidste afsnit af rapporten skrevet.

Analyse

Monolit VS Microservices

Det er allerede givet af problemstillingen, at softwarearkitekturen for "Recipe Recommendation System" skal være microservice-baseret. Men hvorfor egentlig det?

En monolit er én samlet applikation med én kodebase. Den er enkel at udvikle, teste og deploye, fordi al funktionaliteten findes samme sted, og fordi kommunikationen fungerer internt, er der mindre operationel overhead. Et problem opstår, når det skal ændres i koden. Koden er som oftest stramt koblet, og det kan være mere komplekst og tidskrævende at rette i den, og så bliver man nødt til at gen-deploye hele systemet bagefter. Det udgør også en større risiko at rette i koden, fordi en enkelt fejl pga. den stramme kobling kan medføre, at hele systemet går ned. Monolitten har også et problem ift. skalering, fordi når alt er samlet det samme sted, kan man ikke bare opskalere de dele af løsningen, man har brug for.

Tager man Recipe Recommendation Systemet som eksempel, må man antage, at det vil komme mange flere forespørgsler om at hente opskriftsanbefalinger frem for forespørgsler om fx. at oprette opskrifter. Det betyder, at når hele løsningen som monolit skal opskaleres under høj trafik, er det langt mere ressourcekrævende, og ressourcerne bliver ikke udnyttet optimalt, hvilket ikke er særligt omkostningseffektivt.

Designes løsningen med microservice-baseret arkitektur til gengæld, består systemet af uafhængige microservicer, som kan udvikles, testes og deployes selvstændigt. Det medfører dog større operationel overhead, fordi kommunikationen mellem microservicerne sker eksternt over et fælles netværk, og af den grund er det oftest også smart at implementere en form for monitorering til at hjælpe med fejlfinding. Når microservices skal skaleres, er det at foretrække at bruge et orkestreringsværktøj eller en cloudplatform, som tilbyder dynamisk resourceallokering, dvs. opskalere og nedskalere microservicerne efter behov, da det er det mest omkostningseffektive. [3]

Det er derfor en microservice-baseret arkitektur er blevet valgt til projektet som den bedste arkitektur til at lave et stort, skalerbart system.

Bounded Contexts

Efter at have truffet beslutningen om microservice-baseret arkitektur, skal man finde ud af, hvilke Bounded Contexts systemet kan opdeles i, og hvordan disse kan realiseres som microservices.

Systemets to grundlæggende funktionaliteter skal være, at brugere skal modtage opskriftsanbefalinger ud fra ingredienser, og at brugere selv skal være i stand til at bidrage med opskrifter, som andre brugere kan få anbefalet.

Hver funktionalitet har sit eget fokusområde, og hvori det er forskelligt, hvordan man ser på en "Opskrift":

- "Recipe Management"-context: En brugers forespørgsel ændrer i databasens state ved enten at oprette, ændre eller slette opskrifter. Det er alle write-operationer, hvor opskriften skal overholde domæneregler. Konsistens i opskriftsdataen er kritisk, og en "opskrift" ses som en domæneentitet med informationer som fx. ejerskab og oprettelsesdato.
- "Recommendation Read"-context: En brugers forespørgsel læser blot fra databasen, og her er performance vigtigere end øjeblikkelig konsistens i opskriftsdataen, fordi read-operationer blot vil blive kørt på den opskriftsdata, der på det tidspunkt findes i databasen. En "opskrift" ses som et resultat mellem relationer mellem opskrift og ingredienser, og informationer som matchende ingredienser og match-score er vigtige.

Hvordan skal den microservice-baserede arkitektur se ud?

Ud fra den ovenstående identificering af Bounded Contexts skal der i hvert fald være følgende microservices:

- En microservice til at håndtere forespørgsler om oprettelse, ændring og sletning af opskrifter.
- En microservice til at håndtere forespørgsler om at få opskriftsanbefalinger.

Fordi der er tale om en read- og en write-model med tydelige forskelle, er det logisk, at der i hvert fald skal være følgende databaser:

- En database til at tage sig af read-modellen
- En database til at tage sig af write-modellen.

CAP Theorem

CAP Theorem er et begreb, der handler om, at i et distribueret system kan man under netværksfejl kun garantere to ud af tre af følgende: "Consistency", "Availability" og "Partition Tolerance". [4]

Systemet skal være microservice-baseret, og microservices skal kunne fungere uafhængigt af hinanden, selv hvis en skulle gå ned, så "Partition Tolerance" er en forudsætning.

Databasen med read-modellen tåler godt, at der ikke er øjeblikkelig konsistens i opskriftsdataen, fordi det ikke er forretningskritisk, at alle opskrifter findes deri, og der er derfor truffet et bevidst arkitekturvalg om at vælge tilgængelighed (Availability) over øjeblikkelig konsistens (Consistency).

Systemet skal derfor designet sådan, at der opnås eventuel konsistens mellem opskriftsdataen i de to databaser.

Database per service

Lige nu består systemet af:

- En microservice til at håndtere forespørgsler om oprettelse, ændring og sletning af opskrifter.
- En microservice til at håndtere forespørgsler om at få opskriftsanbefalinger.
- En database til at tage sig af read-modellen
- En database til at tage sig af write-modellen.

Sådan som det er nu, skal begge microservices egentlig kommunikere med den database, der har read-modellen; enten for at persistere opskriftsdata eller for at hente opskriftsanbefalinger.

Det er uhensigtsmæssigt, fordi det skaber stram kobling.

Derfor giver det mening at gøre brug af patternet "Database per service", som handler om, at hver microservice skal have deres egen database, som kun de tilgår.

Microservicen, der håndterer forespørgsler om oprettelse, ændring og sletning af opskrifter, er den eneste microservice, der skal kommunikere med databasen med write-modellen, så det giver mening, at den har ejerskab over databasen med write-modellen.

Single Responsibility Principle

Den microservice, der håndterer forespørgsler om at få opskriftsanbefalinger, kan ikke også eje databasen med read-modellen, fordi det vil betyde, at microservicen både vil have ansvar for at persistere opskriftsdata og håndtere opskriftsanbefalinger.

Derfor er det nødvendigt at lave følgende microservice:

- En microservice, som kan eje databasen med read-modellen og har ansvaret for at opbygge og vedligeholde read-modellen baseret på opskriftsdata fra den microservice, der håndterer forespørgsler om oprettelse, ændring og sletning af

opskrifter. Denne microservice håndterer forespørgsler fra de to andre microservices, som vedrører databasen med read-modellen.

Message-drevet arkitektur

For at microservicerne kan kommunikere med hinanden og stadig være løst koblet, er der truffet et valg om at bruge message-drevet arkitektur, hvor microservicerne sender messages til hinanden gennem en message broker. Det er også sådan, systemet sørger for eventuel konsistens i opskriftsdataen ved at sende messages med opskriftsdata fra microservicen, der ejer databasen med write-modellen til den microservice, der ejer databasen med read-modellen.

Asynkront Design og Asynkront Request-Reply Mønster

For at sikre skalerbarhed og løs kobling vil systemet benytte sig af asynkrone kald, så flere processer kan køre samtidig, og ressourcer ikke er låst i længere tid. Som en del af opskriftsanbefalingsflowet vil der blive implementeret asynkront request-reply mønster, hvor et unikt korrelationsid bliver oprettet, når en forespørgsel bliver modtaget, som en frontend efterfølgende kan polle efter et resultat på forespørgslen.[5] I takt med at belastningen på systemet vokser, mere komplekse opskrifter med mange ingredienser bliver tilføjet til databasen, og flere anbefalingsgrundlag fx "kategori" og "bedste bedømmelse" bliver implementeret, forventes det at read-operationerne kan komme til at variere væsentligt i svartid. I stedet for at polle databasen med read-modellen direkte vil der blive benyttet en in-memory database til at cache både status og efterfølgende svaret på forespørgslen for at reducere load på databasen med read-modellen og forbedre applikationens performance.

Polyglot Persistence

Systemet vil gøre brug af mere end en databasetype for at udnytte de styrker, hver valgt database har, og på den måde sikrer den mest hensigtsmæssige håndtering af den type forespørgsler, som databasen modtager.

Teknologivalg

Apache Kafka som message broker

Apache Kafka har mange fordele, som passer godt til Recipe Recommendation System, og derfor er Kafka blevet valgt som message broker. En af fordelene er, at Kafka understøtter horisontal skalering ved at opdele topics i partitions og distribuere dem på tværs af flere brokers, hvilket gør at flere konsumenter kan læse et topic parallelt. Det betyder, at systemets throughput forbliver stabilt i takt med, at systemet bliver større og skal håndtere større workload. Lige nu har systemet kun en konsument på hvert topic, men i fremtiden vil der være flere konsumenter, som deler workloaden, og derfor er det vigtigt, at der allerede nu er valgt en teknologi, hvor det kan lade sig gøre. En anden fordel ved at bruge Kafka er, at Kafka har høj fejltolerance, hvis en node går ned i clusteret. Kafka persisterer nemlig

messages på disk og replicerer dem over alle dens message brokers, så risikoen for datatab er minimal. En tredje fordel er, at Kafka har replayability, dvs. at Kafka kan genspille tidligere messages. Det er rigtig smart ift. Recipe Recommendation System, fordi hvis databasen med read-modellen skulle gå ned, kan den bygges igen ud fra de messages med opskriftsdata, som blev sendt fra microservicen, som håndterer oprettelse, ændring og sletning af opskrifter, og som ejer databasen med write-modellen.[6]

Redis til caching

Den in-memory database som skal cache opskriftsanbefalingsforespørgslens status og svar vil blive implementeret som en Redis database, som er velegnet til midlertidigt data. Redis har meget lav latenstid og kan skaleres horisontalt via Redis cluster, hvilket er vigtigt i et stort system med mange samtidige polling-forespørgsler. Redis understøtter også TTL (time-to-live), hvor cachet data efter et givent tidsinterval automatisk bliver ryddet.[7]

Neo4j som databasen med read-modellen

Databasen med read-modellen vil blive implementeret som en Neo4j grafdatabase, fordi forespørgsler om opskriftsanbefalinger naturligt har en graflignende struktur.

I en Neo4j grafdatabase er opskrifter og ingredienser noder, og deres relationer er edges, som kan traverseres for at finde opskrifter, der matcher forespørgslen fx. find opskrifter, der inkluderer flest ingredienser fra brugerens input.

Det forventes, at langt de fleste forespørgsler til databasen med read-modellen vil være read-operationer, og at der vil være et stort antal af dem, og derfor er der fokus på performance.

Neo4j gør brug af "index-free adjacency", hvor relationerne er lagret direkte i databasen, hvilket gør traversering af relationer hurtigt og skalerbart, fordi det ikke kræver indeks-lookups og join-operationer for at finde den næste relation. En forespørgsel til Neo4j traverserer kun relevante relationer og ignorerer resten af grafen, hvilket betyder, at svartiden ikke direkte bliver påvirket af grafdatabasens størrelse, men af den mængde af relationer der traverseres, hvilket sikrer stabil performance, selv når grafdatabasen vokser. [8][9]

Skulle en relationel database være brugt i stedet for, havde det ført til komplekse join-operationer mellem tabellerne, som er dyre og reducerer performance.

Skulle en dokumentdatabase være brugt i stedet for, havde det ført til omfattende dataduplikering for at repræsentere relationerne, som havde været svært at vedligeholde.

MongoDB som databasen med write-modellen

Databasen med write-modellen vil blive implementeret som en MongoDB, fordi en MongoDB har indbygget horisontal skalering, som er en mere hensigtsmæssig måde at skalere på frem for vertikal skalering, som en relationel database traditionelt lægger op til. Lige nu har

MongoDB'en ikke brug for at blive skaleret op, men i takt med at der kommer flere brugere af systemet, forventes det at blive nødvendigt.

MongoDB kan skaleres horisontalt via sharding, hvor datasættet deles ud på flere servere i et cluster. Hvert shard er et replica sets, hvor én node fungerer som primærnoden og alene håndterer write-operationer, og så bliver dataen repliceret af de andre noder for at øge fejltolerance og tilgængelighed. Hvis primærnoden går ned, bliver en ny primærnode valgt.[10]

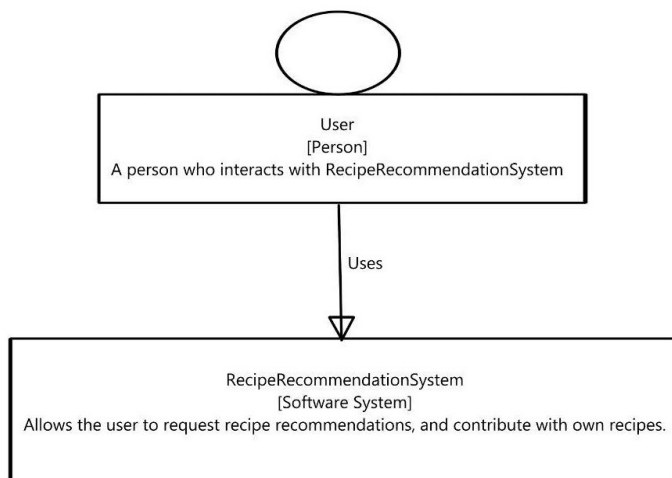
Skulle en relationel database være brugt i stedet for, havde horisontal skalering krævet kompleks manuel sharding eller brug af tredjepartsværktøjer, og man havde i applikationslogikken måtte implementere på hvilken databaseinstans bestemt data findes.

Design

C4-modellen til softwarearkitektur

C4-modellen er valgt for visuelt at vise nogle af de vigtige designbeslutninger, der er truffet ift., hvordan microservicerne skal kommunikere med hinanden og de andre docker containere i systemet.[11]

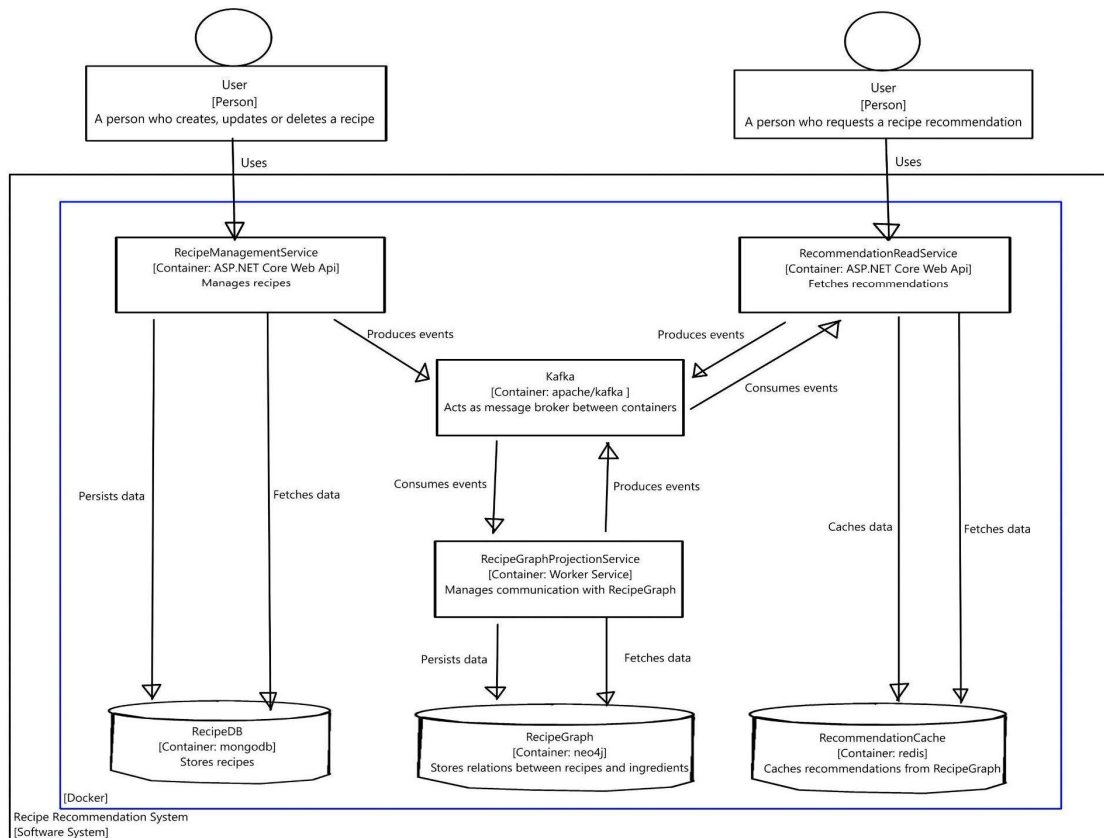
Level 1: System Context Diagram for RecipeRecommendationSystem



Formålet med at lave et system context diagram er at vise systemets omgivelser, typisk hvordan det indgår sammen med andre systemer i en større forretningsproces. Der vises også systemets primære aktører og deres forudsætninger for at bruge systemet.

RecipeRecommendationSystem er et proof-of-concept system i et teknologistudie og er tiltænkt som et selvstændigt system, som ikke indgår i en større forretningsproces. En bruger er på nuværende tidspunkt blot karakteriseret som en person, der bruger systemet, uden en brugerkonto.

Level 2: Container Diagram for RecipeRecommendationSystem



Formålet med at lave et container diagram er at vise de containere, der er i systemet, og de teknologier, der bliver brugt.

Systemet består af syv docker containere, hvoraf tre er microservices, tre er databaser, og én er message broker. Hver container har sit eget ansvarsområde og kan selvstændigt deployes. For at sikre løs kobling, er der fokuseret på message-drevet design mellem microservicerne, og derfor sker kommunikationen mellem dem via message-broderen Kafka. Der er også lagt vægt på asynkront design for skalerbarhed og vedligeholdelse, så hver kald mellem containerne og inde i hver container er asynkront.

For læsevenlighedsens skyld og projektets tidsbegrænsning, viser dette diagram kun de containere, der matcher det scope, der er sat for projektet, og dermed kun de containere, der vil blive implementeret. Et ideelt container diagram vil vise hele systemet.

En bruger har - via en fremtidigt implementeret brugergrænseflade og API Gateway - to indgangsvinkler til systemet, enten gennem containeren **RecipeManagementService** eller gennem containeren **RecommendationReadService**.

Containeren **RecipeManagementService** er et [ASP.NET](#) Core Web Api, som har ansvar for at håndtere forespørgsler fra brugeren, når brugeren ønsker at oprette, opdatere eller slette egne opskrifter. **RecipeManagementService** har også til ansvar at persistere opskriftsdata i **RecipeDB**-databasen og via message-broderen **Kafka** sende messages med opskriftsdataen til containeren **RecipeGraphProjectionService**, hvor opskriften også bliver persisteret for at

sikre konsistens mellem opskrifterne i RecipeDB og opskrifterne i RecipeGraph. Det er kun endpoints for POST, UPDATE og DELETE, der er tilgængelige for brugeren i denne container.

Containeren RecipeDB er en container bygget ud fra et MongoDB docker image. RecipeDB har til ansvar at persistere opskrifter, som kommer fra RecipeManagementService. RecipeDB modtager kun write commands fra RecipeManagementService på brugerens vegne. Det er ikke RecipeDB's ansvar at generere opskriftsanbefalinger, så det er ikke her, hvor opskriftsdata fetches til brugeren. Det er dog muligt at fetche hhv. en specifik opskrift og alle opskrifter for en databaseadministrator.

Containeren Kafka er en container bygget ud fra et Apache Kafka docker image. Kafka er en message-broker og har ansvaret for at route messages fra/til følgende containere i systemet.

- Fra RecipeManagementService til RecipeGraphProjectionService: Herimellem sendes messages vedrørende en oprettet/ændret/slettet opskrift.
- Fra RecommendationReadService til RecipeGraphProjectionService: Herimellem sendes messages vedrørende forespørgsler på opskriftsanbefalinger.
- Fra RecipeGraphProjectionService til RecommendationReadService: Herimellem sendes messages vedrørende svaret på opskriftsanbefalingerne.

Kafka har ansvaret at håndtere oprettelse og administration af topics, lytte efter messages publiceret af producers og distribuerer dem til de consumers, der er subscriberet på de relevante topics.

Containeren RecommendationReadService er et [ASP.NET](#) Core Web Api, som har ansvar for at håndtere forespørgsler fra brugeren, når brugeren ønsker at få anbefalet opskrifter ud fra ingredienser, brugeren har indtastet. RecommendationReadService har også til ansvar at cache opskriftsanbefalingsdata i RecommendationCache databasen for at sikre, at forespørgslen bliver koblet sammen med den rigtige respons. RecommendationReadService sender via message-broderen Kafka messages med opskriftsanbefalingsdataen til containeren RecipeGraphProjectionService, hvor forespørgslen om opskriftsanbefalinger bliver videre behandlet. RecommendationReadService lytter kontinuerligt efter messages fra RecipeGraphProjectionService med respons på opskriftsanbefalingsforespørgsler.

Containeren RecommendationCache er en container bygget ud fra et Redis docker image. RecommendationCache har til ansvar at eksekvere de queries, der kommer fra RecommendationReadService, som cacher opskriftsanbefalingsdata.

Containeren RecipeGraphProjectionService er en worker service, som kører i baggrunden og har til ansvar kontinuerligt at lytte efter messages fra enten RecipeManagementService eller RecommendationReadService, som den kan konsumere, så den kan håndtere forespørgslerne fra de to andre containere og kører de passende queries på RecipeGraph databasen. Når RecipeGraphProjectionService konsumerer messages fra RecipeManagementService, bliver opskriftsdataen blot persisteret i RecipeGraph databasen, og RecipeManagementService forventer ikke en message returneret som svar på, at forespørgslen er en succes. Når RecipeGraphProjectionService konsumerer messages fra RecommendationReadService, danner RecommendationReadService sammen med RecommendationGraphProjectionService et request/response flow, hvor

RecommendationReadService forventer at få en message med responsen på opskriftsanbefalingsforespørgslen returneret.

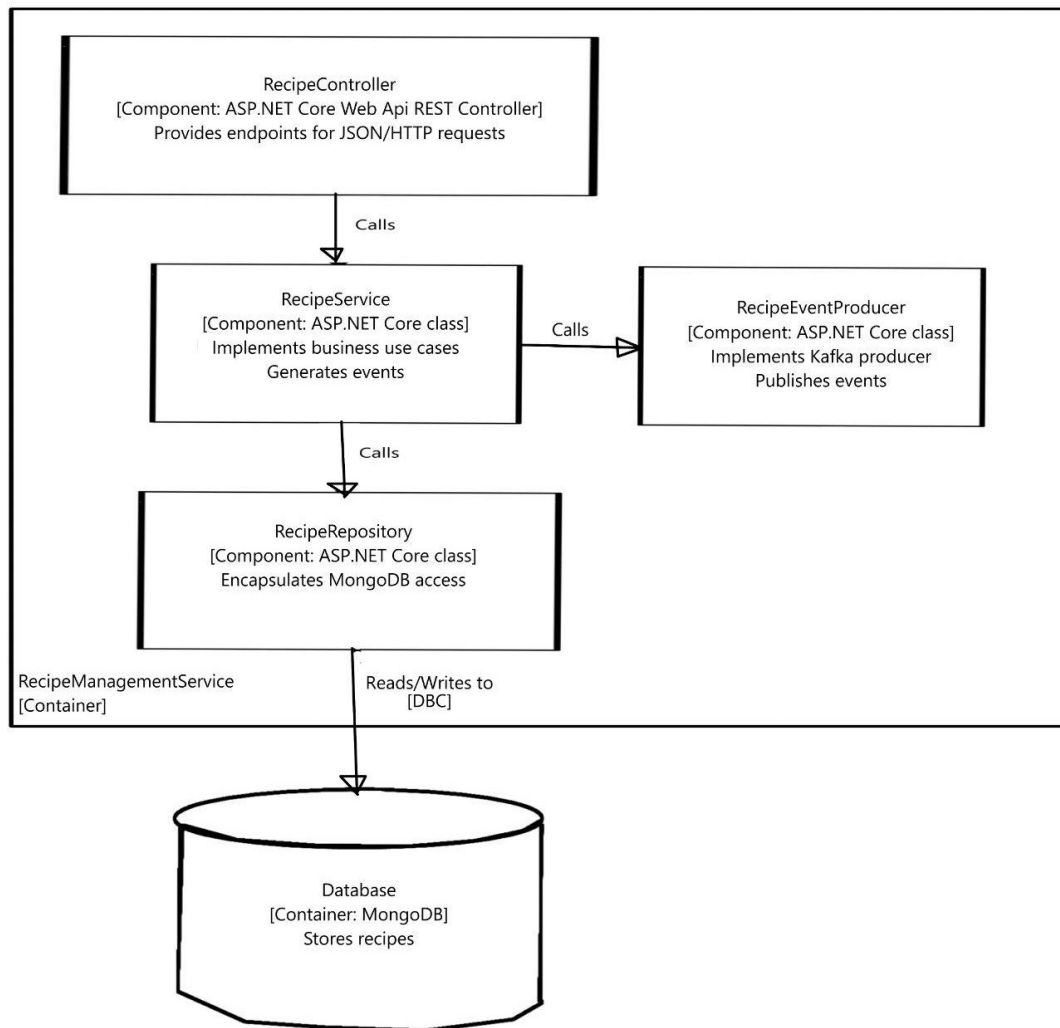
Containeren RecipeGraph er en container bygget ud fra et Neo4j docker image. RecipeGraph har til ansvar at eksekvere de queries, der kommer fra RecipeGraphProjectionService, og returnere responsen dertil.

Level 3: Component diagram for alle microservicer i RecipeRecommendationSystem

Level 3 i C4-modellen er component diagram, som giver et mere detaljeret indblik i softwarearkitekturen ved at vise den interne struktur i hver container. Der vises de logiske komponenter hver container består af, og hvordan komponenterne kommunikerer internt med hinanden, og hvilke kald der går ind og ud af containeren.

Der er valgt at lave et component diagram for hver microservice, da de er designet fra bunden, mens databaserne og message-brokeren har en fastsat struktur, der ligger uden for projektets scope.

Component Diagram for RecipeManagementService



I containeren RecipeManagementService findes disse komponenter:

Komponenten RecipeController har ansvar for at eksponere REST-endpoints og håndterer den HTTP request, brugeren sender, når brugeren ønsker at oprette, opdatere eller slet egne opskrifter. Når en forespørgsel er modtaget, kalder RecipeController på RecipeService og sender information fra forespørgslen ned.

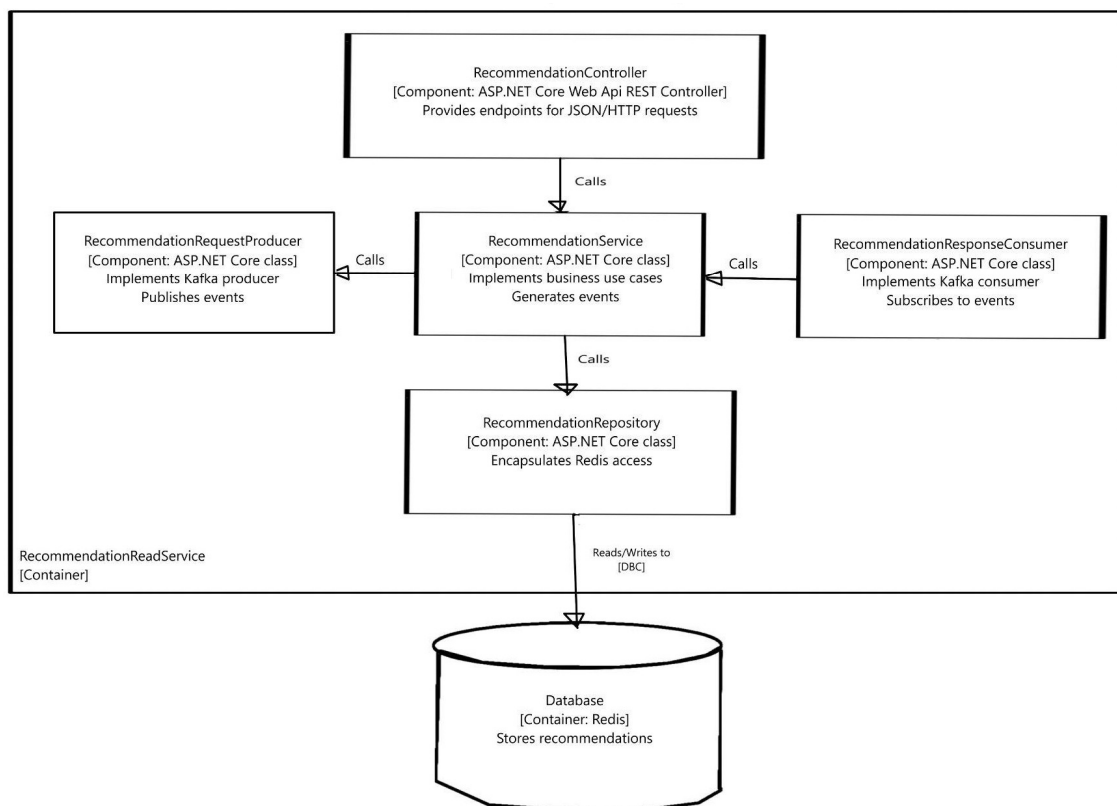
Komponenten RecipeService fungerer som applikationsservice og har ansvar for håndtering af forretningsmæssige use cases for opskrifter og producerer tilsvarende messages, så andre containere er oplyst om brugerens handling. Use cases "Create Recipe", "Update Recipe" og "Delete Recipe" bliver håndteret på samme måde og har samme flow gennem containeren. Først kalder RecipeService på RecipeRepository. Efterfølgende generer RecipeService en message, der beskriver, hvilken type request brugeren har lavet med relevant opskriftsdata inkluderet. Derefter kalder RecipeService på RecipeEventProducer.

Komponenten RecipeEventProducer implementerer en Kafka producer og har til ansvar at publicere messagen til Kafka under den rigtige topic, så microservicen

RecipeGraphProjectionService kan få besked om forespørgsler på at oprette/opdatere/slette en opskrift.

Komponenten RecipeRepository har ansvar for at skabe forbindelse til og skrive til MongoDB-databasen, så oprettelsen/ændringen/sletningen af en opskrift bliver persisteret. RecipeRepository er den komponent alene, der kan sende forespørgsler direkte til MongoDB-databasen.

Component Diagram for RecommendationReadService



Containeren RecommendationReadService har følgende komponenter:

Komponenten RecommendationController har ansvaret for at eksponere REST-endpoints og håndterer den HTTP request, brugeren sender, når brugeren ønsker opskriftsanbefalinger ud fra ingrediens input. RecommendationController kalder på RecommendationService.

Komponenten RecommendationService fungerer som applikationsservice og er ansvarlig for at håndtere forretningsmæssige use cases for opskriftsanbefalinger og producerer tilsvarende messages, så andre containere er oplyst om brugerens handling. Når en forespørgsel modtages, genererer RecommendationService en unik identifikator for den specifikke forespørgsel og sætter forespørgslens behandlingsstatus som igangværende. RecommendationService sender derefter oplysningerne ned til RecommendationRepository.

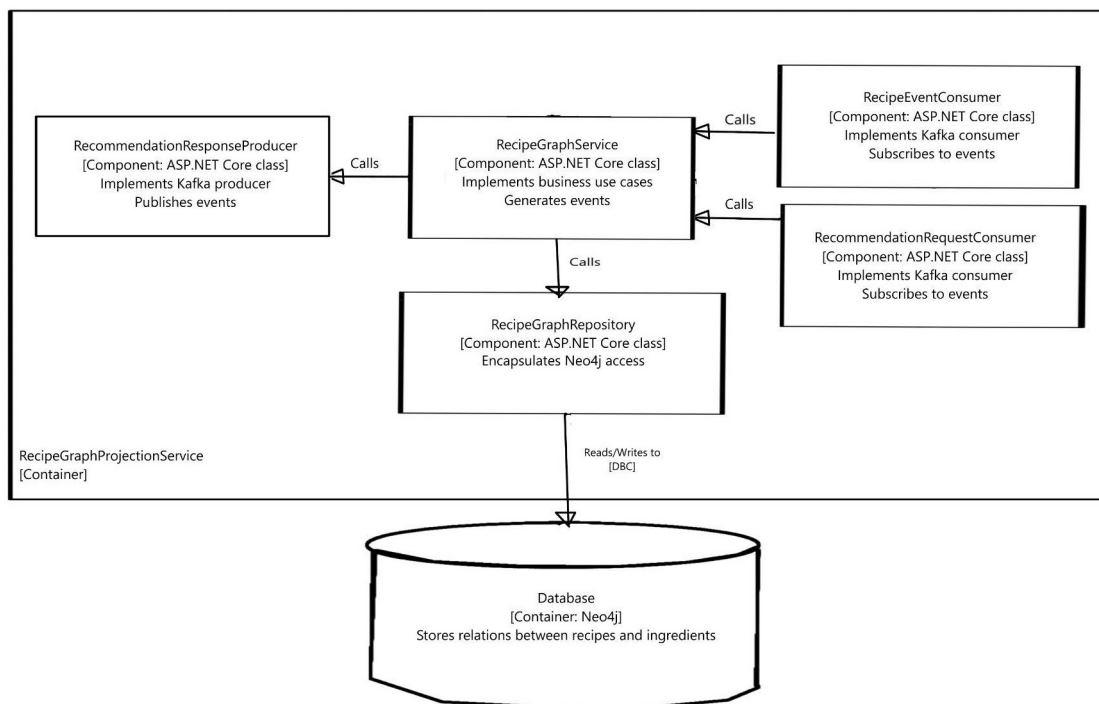
Efterfølgende genererer RecommendationService en message med den unikke identifikator og ingredienserne fra brugerens input og kalder på RecommendationRequestProducer.

Komponenten RecommendationRequestProducer implementerer en Kafka producer og har til ansvar at publicere messages til Kafka, så microservicen RecipeGraphProjectionService kan få besked om forespørgsler på opskriftsanbefalinger.

Komponenten RecommendationRepository er ansvarlig for forbindelse til og kommunikation med Redis databasen og er den komponent alene, der kan sende forespørgsler direkte til Redis databasen. RecommendationRepository er ansvarlig for at cache en forespørgsels unikke identifikator og igangværende behandlingsstatus, når forespørgslen først bliver modtaget af microservicen. RecommendationRepository er også ansvarlig for at cache den responsdata, som RecommendationResponseConsumer modtager, så et færdigt respons med anbefalede opskrifter kan fetches til brugeren.

Komponenten RecommendationResponseConsumer implementerer en Kafka consumer og har ansvaret for at lytte efter messages via Kafka fra RecommendationResponseProducer i RecipeGraphProjectionService og sende responsdataen til RecommendationService.

Component Diagram for RecipeGraphProjectionService



I containeren RecipeGraphProjectionService er der følgende komponenter:

Komponenten RecipeEventConsumer implementerer en Kafka consumer, og har til ansvar at konsumere de messages, som RecipeEventProducer fra RecipeManagementService har

sendt til Kafka, og at få den data, som disse messages indeholder, sendt til RecipeGraphService.

Komponenten RecommendationRequestConsumer implementerer ligeledes en Kafka consumer, og er ansvarlig for at konsumere de messages, som RecommendationRequestProducer fra RecommendationReadService har sendt til Kafka og ligeledes få den data, som disse messages indeholder, sendt til RecipeGraphService.

Komponenten RecipeGraphService fungerer som applikationsservice og står for at implementere de forretningsmæssige use cases, som også fandtes i Web Api'et RecipeManagementService og Web Api'et RecommendationReadService samt publicerer tilsvarende messages, der holder andre container i systemet oplyst. RecipeGraphService's ansvar er at koordinere behandlingen af forespørgsler fra de to Web Api'er. Når RecipeGraphService har modtaget en forespørgsel, sendes den videre til RecipeGraphRepository for at blive besvaret. Når der er svar på en forespørgsel, er det RecipeGraphService, der generer en message med svaret og kalder på RecommendationResponseProducer.

Komponenten RecipeGraphRepository er ansvarlig for forbindelsen og kommunikationen til Neo4j-databasen, og det er den komponent alene, der kan sende forespørgsler direkte til Neo4j-databasen. Både forespørgsler om at persistere oprettelse, opdatering eller sletning af opskrifter samt forespørgsler om at fetche opskriftsanbefalinger ud fra bestemte ingredienser bliver håndteret her via queries til databasen.

Komponenten RecommendationResponseProducer implementerer en Kafka producer og har ansvar for at publicere messages til Kafka, så svaret på en forespørgsel om opskriftsanbefalinger via Kafka kan blive returneret til microservicen RecommendationReadService.

Implementering af systemet

I dette tilfælde bliver requests fra brugeren simuleret gennem Postman, men i det færdige system vil requests komme fra brugergrensefladen, der er til rådighed for brugeren.

For at vise flowet gennem systemet vil jeg bruge de forretningsmæssige use cases "Create Recipe" og "Get Recommendation".

Flowet for "Create Recipe"

RecipeManagementService

RecipeController

I RecipeControlleren tager CreateRecipe imod et Recipe-objekt som JSON fra request bodyen, tjekker at ModelState er valid, sender objektet videre til RecipeService-klassen, og returnerer til sidst en 201 statuskode, hvis requesten er en succes.

```

38 // POST: api/Recipe
39 [HttpPost]
40 0 references
41 public async Task<IActionResult> CreateRecipe([FromBody] Recipe recipe)
42 {
43     if (!ModelState.IsValid)
44     {
45         return BadRequest(ModelState);
46     }
47     await _recipeService.CreateRecipe(recipe);
48     return CreatedAtAction(nameof(GetRecipeByRecipeId), new { recipeId = recipe.RecipeId }, recipe);
49 }

```

RecipeService

I RecipeService sendes recipe videre til RecipeRepository-klassen. Derefter bliver der lavet et event med den oprettede opskriffs oplysninger, og en RecipeEventProducer sender eventet til en dockeriseret Kafka container under topic'en "recipe-created".

```

30 public async Task CreateRecipe(Recipe recipe)
31 {
32     await _recipeRepository.CreateRecipe(recipe);
33
34     var message = new RecipeCreatedEvent
35     {
36         RecipeId = recipe.RecipeId,
37         Name = recipe.Name,
38         Description = recipe.Description,
39         Ingredients = recipe.Ingredients,
40         Instructions = recipe.Instructions,
41         Category = recipe.Category
42     };
43
44     await _recipeEventProducer.ProduceAsync("recipe-created", new Message<string, string>
45     {
46         Key = recipe.RecipeId,
47         Value = JsonSerializer.Serialize(message)
48     });
49 }

```

RecipeRepository

I RecipeRepository bliver Recipe persisteret i kollektionen "Recipes" i MongoDB-databasen "RecipeDB".

```

29 public async Task CreateRecipe(Recipe recipe)
30 {
31     await _recipesCollection.InsertOneAsync(recipe);
32 }
33

```

En Recipe ser ud på denne måde.

```

6      public class Recipe
7      {
8          [BsonId] // MongoDB _id field
9          [BsonRepresentation(BsonType.ObjectId)]
10         public string? RecipeId { get; set; }
11
12         [BsonElement("name")]
13         public string Name { get; set; } = null!;
14
15         [BsonElement("description")]
16         public string Description { get; set; } = null!;
17
18         [BsonElement("ingredients")]
19         public List<string> Ingredients { get; set; } = new List<string>();
20
21         [BsonElement("instructions")]
22         public List<string> Instructions { get; set; } = new List<string>();
23
24         [BsonElement("category")]
25         public string Category { get; set; } = null!;
26     }
27 }

```

RecipeGraphProjectionService

RecipeEventConsumer

RecipeEventConsumer lytter på Kafka-topics "recipe-created", "recipe-updated" og "recipe-deleted", som tilhører consumer gruppen "recipe-projection-group". Hvis et event med topic'en "recipe-created" modtages, bliver det deserialiseret til event-objektet RecipeCreatedEvent og videresendes til RecipeGraphService, som opretter Recipe i graf-databasen.

```

7  namespace RecipeGraphProjectionService.Infrastructure.Messaging
8  {
9      6 references
10     public class RecipeEventConsumer : IRecipeEventConsumer
11     {
12         private readonly IRecipeGraphService _recipeGraphService;
13         private readonly IConsumer<string, string> _consumer;
14         private readonly ILogger<RecipeEventConsumer> _logger;
15
16         0 references
17         public RecipeEventConsumer(
18             IRecipeGraphService recipeGraphService,
19             ILogger<RecipeEventConsumer> logger)
20         {
21             _recipeGraphService = recipeGraphService;
22             _logger = logger;
23
24             var config = new ConsumerConfig
25             {
26                 GroupId = "recipe-projection-group",
27                 BootstrapServers = "kafka:29092",
28                 AutoOffsetReset = AutoOffsetReset.Earliest
29             };
30
31             _consumer = new ConsumerBuilder<string, string>(config).Build();
32
33         2 references
34         public Task StartAsync(CancellationToken token)
35         {
36             _consumer.Subscribe(new[]
37             {
38                 "recipe-created",
39                 "recipe-updated",
40                 "recipe-deleted"
41             });
42
43             return Task.Run(async () =>
44             {
45                 while (!token.IsCancellationRequested)
46                 {
47                     var result = _consumer.Consume(token);
48                     _logger.LogInformation("Event received on topic {Topic}", result.Topic);
49                     await DispatchEventAsync(result.Topic, result.Message.Value);
50                 }
51             }, token);
52         }
53
54         1 reference
55         private async Task DispatchEventAsync(string topic, string message)
56         {
57             switch (topic)
58             {
59                 case "recipe-created":
60                     var created = JsonSerializer.Deserialize<RecipeCreatedEvent>(message,
61                         new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
62                     _logger.LogInformation("RAW EVENT: " + message);
63                     await _recipeGraphService.CreateRecipeAsync(
64                         created!.RecipeId,
65                         created.Name,
66                         created.Description,
67                         created.Ingredients,
68                         created.Instructions,
69                         created.Category
70                     );
71                     break;
72             }
73         }
74     }
75

```

RecipeGraphService

RecipeGraphService modtager event-objektet og kalder på RecipeGraphRepository's create metode.

```

10 namespace RecipeGraphProjectionService.Application.Services
11 {
12     4 references
13     public class RecipeGraphService : IRecipeGraphService
14     {
15         private readonly IRecipeGraphRepository _repository;
16         private readonly IRecommendationResponseProducer _recommendationResponseProducer;
17         private readonly ILogger<RecipeGraphService> _logger;
18
19         0 references
20         public RecipeGraphService(
21             IRecipeGraphRepository repository,
22             IRecommendationResponseProducer recommendationResponseProducer,
23             ILogger<RecipeGraphService> logger)
24         {
25             _repository = repository;
26             _recommendationResponseProducer = recommendationResponseProducer;
27             _logger = logger;
28
29         2 references
30         public async Task CreateRecipeAsync(
31             string recipeId,
32             string name,
33             string description,
34             List<string> ingredients,
35             List<string> instructions,
36             string category)
37         {
38             await _repository.CreateRecipeAsync(
39                 recipeId,
40                 name,
41                 description,
42                 ingredients,
43                 instructions,
44                 category
45             );
46             _logger.LogInformation("Recipe {Id} created in Neo4j graph.", recipeId);
47         }

```

RecipeGraphRepository

I RecipeGraphRepository oprettes der en recipe-node med den medsendte data. Herefter merges hver ingrediens som en Ingredient-node, og der oprettes en relation mellem opskriften og ingredienserne af typen USES. Merging sikrer, at ingredienserne kan genbruges på tværs af flere opskrifter.


```

16 public async Task CreateRecipeAsync(
17     string recipeId,
18     string name,
19     string description,
20     List<string> ingredients,
21     List<string> instructions,
22     string category)
23 {
24     await using var session = _driver.AsyncSession();
25
26     await session.ExecuteWriteAsync(async tx =>
27     {
28         // Create the recipe and bind it
29         var cursor = await tx.RunAsync(
30             @"MERGE (r:Recipe {id: $id})
31             SET r.name = $name,
32                 r.description = $description,
33                 r.instructions = $instructions,
34                 r.category = $category
35             RETURN r",
36             new
37             {
38                 id = recipeId,
39                 name,
40                 description,
41                 instructions = instructions.ToArray(),
42                 category
43             }
44         );
45
46         var recipeNode = await cursor.SingleAsync();
47
48         // Reuse recipe node - DO NOT MERGE AGAIN
49         await tx.RunAsync(
50             @"UNWIND $ingredients AS ingredientName
51             MERGE (i:Ingredient {name: ingredientName})
52             WITH i
53             MATCH (r:Recipe {id: $id})
54             MERGE (r)-[:USES]->(i)",
55             new
56             {
57                 id = recipeId,
58                 ingredients
59             }
60         );
61     });
62 }

```

Flowet for “Get Recommendation”

RecommendationReadService

RecommendationController

RecommendationController tager imod en post-request med en liste af ingredienser i request-bodyen. Controlleren tjekker, om listen er tom. Hvis ikke, så sendes listen af ingredients som parameter til RecommendationService, hvorefter den modtager et correlationId, der bliver brugt til at spore anbefalingforespørgslen på tværs af services. Herefter returneres id’et og status som er “PROCESSING”.


```

17 // POST: api/Recommendation
18 [HttpPost]
19 // 0 references
20 public async Task<IActionResult> RequestRecommendations([FromBody] List<string> ingredients)
21 {
22     try
23     {
24         if (ingredients == null || ingredients.Count == 0)
25             return BadRequest("Ingredients cannot be empty.");
26
27         var correlationId = await _recommendationService.StartRecommendationRequest(ingredients);
28
29         Console.WriteLine("After call to StartRecommendationRequest");
30
31         return Accepted(new
32         {
33             correlationId,
34             status = "PROCESSING"
35         });
36     }
37     catch (Exception ex)
38     {
39         // Log exception
40         Console.WriteLine(ex);
41         return StatusCode(500, ex.Message);
42     }
43 }

```

RecommendationService

Når RecommendationService bliver kaldt af RecommendationController med en liste af ingredienser, genererer den først et unikt correlationId, som bliver brugt i det nye RecommendationStateDto objekt, som bliver kreeret med correlationId, status og Recipes, som er sat til null. DTO'en sendes videre sammen med correlationId til RecommendationRepository. Herefter oprettes et event med correlationId og ingredienslisten, som bliver sendt til Kafka via RecommendationProducer med topic'et "recommendation-request" og der returneres correlationId og status.


```

11 namespace RecipeGraphProjectionService.Infrastructure.Messaging
12 {
13     2 references
14     public class RecommendationRequestConsumer : IRecommendationRequestConsumer
15     {
16         private readonly IRecipeGraphService _recipeGraphService;
17         private readonly IConsumer<string, string> _consumer;
18         private readonly ILogger<RecipeEventConsumer> _logger;
19
20         0 references
21         public RecommendationRequestConsumer(
22             IRecipeGraphService recipeGraphService,
23             ILogger<RecipeEventConsumer> logger)
24         {
25             _recipeGraphService = recipeGraphService;
26             _logger = logger;
27
28             var config = new ConsumerConfig
29             {
30                 GroupId = "recommendation-group",
31                 BootstrapServers = "kafka:29092",
32                 AutoOffsetReset = AutoOffsetReset.Earliest
33             };
34
35             _consumer = new ConsumerBuilder<string, string>(config).Build();
36
37         2 references
38         public Task StartAsync(CancellationToken token)
39         {
40             _consumer.Subscribe(new[]
41             {
42                 "recommendation-request"
43             });
44
45             return Task.Run(async () =>
46             {
47                 while (!token.IsCancellationRequested)
48                 {
49                     var result = _consumer.Consume(token);
50                     _logger.LogInformation("Event received on topic {Topic}", result.Topic);
51                     await DispatchEventAsync(result.Topic, result.Message.Value);
52                 }
53             }, token);
54         }
55
56         1 reference
57         private async Task DispatchEventAsync(string topic, string message)
58         {
59             switch (topic)
60             {
61                 case "recommendation-request":
62                     var request = JsonSerializer.Deserialize<RecommendationRequestEvent>(message);
63                     await _recipeGraphService.GetRecommendedRecipesAsync(request.CorrelationId, request.Ingredients);
64                     Console.WriteLine("Message received by RecommendationRequestConsumer");
65                     break;
66             }
67         }
68     }
69 }
70

```

RecipeGraphService

I RecipeGraphService modtages correlationId og ingredienser. Ingredienslisten bliver sendt videre til RecipeGraphRepository for at hente anbefalede opskrifter. Efter at have modtaget de tre mest matchende opskrifter bliver opskrifter sammen med correlationId lavet til et RecommendationResponseEvent og sendt via RecommendationResponseProducer til Kafka med topic'et "recommendation-response".

```
48 | 2 references
49 | public async Task GetRecommendedRecipesAsync(string correlationId, List<string> ingredientsFromFridge)
50 | {
51 |     var recommended = await _repository.GetRecommendedRecipesAsync(ingredientsFromFridge);
52 |
53 |     var response = new RecommendationResponseEvent
54 |     {
55 |         CorrelationId = correlationId,
56 |         RecommendedRecipes = recommended
57 |     };
58 |
59 |     var json = JsonSerializer.Serialize(response);
60 |
61 |     await _recommendationResponseProducer.ProduceAsync(
62 |         "recommendation-response",
63 |         new Message<string, string>
64 |         {
65 |             Key = correlationId,
66 |             Value = json
67 |         });
68 |     _logger.LogInformation("Sent recommendation response for correlation {CorrelationId}", correlationId);
69 | }
```

RecipeGraphRepository

I RecipeGraphRepository matches opskrifterne med ingredienser ved at bruge graf-relationer mellem Recipe- og Ingredient-nodes i Neo4j-databasen. For hver opskrift kalkuleres antallet af matchende ingredienser, manglende ingredienser samt en score til at sortere, hvilke opskrifter der matcher mest. De tre mest matchende opskrifter laves til recommendedRecipeDto objekter, som returneres til RecipeGraphService.

```

71 2 references
72 public async Task<List<RecommendedRecipeDto>> GetRecommendedRecipesAsync(List<string> ingredientsFromFridge)
73 {
74     await using var session = _driver.AsyncSession();
75
76     return await session.ExecuteReadAsync(async tx =>
77     {
78         var cursor = await tx.RunAsync(
79             @"MATCH (r:Recipe)-[:USES]->(i:Ingredient)
80             WITH r, collect(i.name) AS recipeIngredients,
81                 $ingredientsFromFridge AS ingredientsFromFridge
82             WITH r,
83                 recipeIngredients,
84                 [x IN recipeIngredients WHERE x IN ingredientsFromFridge] AS matched,
85                 [x IN recipeIngredients WHERE NOT x IN ingredientsFromFridge] AS missing
86             WHERE size(matched) > 0
87             RETURN r.id AS recipeId,
88                 r.name AS name,
89                 r.description AS description,
90                 r.instructions AS instructions,
91                 r.category AS category,
92                 recipeIngredients,
93                 matched,
94                 missing,
95                 size(matched) AS matchCount,
96                 size(recipeIngredients) AS totalIngredients,
97                 (toFloat(size(matched)) / size(recipeIngredients)) AS score
98             ORDER BY score DESC, size(missing) ASC
99             LIMIT 3",
100             new { ingredientsFromFridge }
101         );
102
103         var results = new List<RecommendedRecipeDto>();
104
105         await cursor.ForEachAsync(record =>
106         {
107             results.Add(new RecommendedRecipeDto
108             {
109                 RecipeId = record["recipeId"].As<string>(),
110                 Name = record["name"].As<string>(),
111                 Description = record["description"].As<string>(),
112                 Instructions = record["instructions"].As<List<object>>>().Cast<string>().ToList(),
113                 Category = record["category"].As<string>(),
114                 RecipeIngredients = record["recipeIngredients"].As<List<object>>>().Cast<string>().ToList(),
115                 MatchedIngredients = record["matched"].As<List<object>>>().Cast<string>().ToList(),
116                 MissingIngredients = record["missing"].As<List<object>>>().Cast<string>().ToList(),
117                 MatchCount = record["matchCount"].As<int>(),
118                 TotalIngredients = record["totalIngredients"].As<int>(),
119                 Score = record["score"].As<double>()
120             });
121         });
122
123         return results;
124     });

```

RecommendationReadService

RecommendationResponseConsumer

Når RecommendationResponseConsumer modtager et event fra Kafka topic'et "recommendation-response" deserialiseres det til et RecommendationResponseEvent objekt, hvor correlationId og listen af anbefalede opskrifter (recommendRecipes) sendes til RecommendationService.

```

6 namespace RecommendationReadService.Infrastructure.Kafka
7 {
8     4 references
9     public class RecommendationResponseConsumer : BackgroundService
10     {
11         private readonly IRecommendationService _recommendationService;
12         private readonly IConsumer<string, string> _consumer;
13         private readonly ILogger<RecommendationResponseConsumer> _logger;
14
15         0 references
16         public RecommendationResponseConsumer(
17             IRecommendationService recommendationService,
18             ILogger<RecommendationResponseConsumer> logger)
19         {
20             _recommendationService = recommendationService;
21             _logger = logger;
22
23             var config = new ConsumerConfig
24             {
25                 GroupId = "recommendation-group",
26                 BootstrapServers = "kafka:29092",
27                 AutoOffsetReset = AutoOffsetReset.Earliest
28             };
29
30             _consumer = new ConsumerBuilder<string, string>(config).Build();
31
32         }
33
34         0 references
35         protected override async Task ExecuteAsync(CancellationToken stoppingToken)
36         {
37             _consumer.Subscribe("recommendation-response");
38             _logger.LogInformation("Kafka consumer subscribed.");
39
40             while (!stoppingToken.IsCancellationRequested)
41             {
42                 try
43                 {
44                     var result = _consumer.Consume(TimeSpan.FromMilliseconds(200));
45
46                     if (result != null)
47                     {
48                         await HandleMessageAsync(result.Message.Value);
49                     }
50                 }
51                 catch (ConsumeException ex)
52                 {
53                     _logger.LogError(ex, "Kafka consume error");
54                 }
55
56                 await Task.Delay(50, stoppingToken);
57             }
58
59             1 reference
60             private async Task HandleMessageAsync(string message)
61             {
62                 try
63                 {
64                     var response = JsonSerializer.Deserialize<RecommendationResponseEvent>(message);
65                     if (response != null)
66                     {
67                         await _recommendationService.CacheRecommendationResponse(response);
68                     }
69                 }
70                 catch (Exception ex)
71                 {
72                     _logger.LogError(ex, "Error processing Kafka message");
73                 }
74             }
75         }
76     }
77 }

```

RecommendationService

I RecommendationService omdannes correlationId og recommendedRecipes til RecommendationStateDto objekt, hvor feltet status er nu sat til "COMPLETE". Dette objekt sendes til RecommendationRepository sammen med correlationId for at blive cachet.


```

70 |         public async Task CacheRecommendationResponse(string correlationId, List<RecommendedRecipeDto> recommendedRecipes)
71 |         {
72 |             var completedState = new RecommendationStateDto
73 |             {
74 |                 CorrelationId = correlationId,
75 |                 Status = "COMPLETE",
76 |                 Recipes = recommendedRecipes
77 |             };
78 |             await _recommendationRepository.SaveRecommendationState(correlationId, completedState);
79 |         }
80 |     }

```

RecommendationRepository

I RecommendationRepository caches RecommendationStateDto sammen med correlationId.

```

28 |         public async Task SaveRecommendationState(string correlationId, RecommendationStateDto state)
29 |         {
30 |             await _redis.AddAsync(RedisKeyPrefix + correlationId, state, RedisTTL);
31 |         }

```

RecommendationController

Når RecommendationController modtager et GET request til om at fetche resultatet på opskriftsanbefalingsforespørgslen, kalder den på RecommendationService klassen med correlationId. Hvis der findes noget på id'et så returneres der statuskode 200 og objektet.

```

45 |         // GET: api/Recommendation/{correlationId}
46 |         [HttpGet("{correlationId}")]
47 |         public async Task<IActionResult> GetRecommendationResult(string correlationId)
48 |         {
49 |             var state = await _recommendationService.GetRecommendationStatus(correlationId);
50 |
51 |             if (state == null)
52 |                 return NotFound("Unknown correlation ID");
53 |
54 |             return Ok(state);
55 |         }

```

RecommendationService

I RecommendationService sendes forespørgslen videre til RecommendationRepository.

```

8 | namespace RecommendationReadService.Application.Services
9 | {
10 |     public class RecommendationService : IRecommendationService
11 |     {
12 |         private readonly IRecommendationRequestProducer _recommendationRequestProducer;
13 |         private readonly IRecommendationRepository _recommendationRepository;
14 |
15 |         public RecommendationService(IRecommendationRequestProducer recommendationRequestProducer, IRecommendationRepository recommendationRepository)
16 |         {
17 |             _recommendationRequestProducer = recommendationRequestProducer;
18 |             _recommendationRepository = recommendationRepository;
19 |         }
20 |
21 |         // STEP 2 - Check recommendation status (polling from controller)
22 |         //
23 |         public async Task<RecommendationStateDto?> GetRecommendationStatus(string correlationId)
24 |         {
25 |             return await _recommendationRepository.GetRecommendationStatus(correlationId);
26 |         }
27 |     }

```

RecommendationRepository

I RecommendationRepository bliver RecommendationStateDto hentet baseret på correlationId'et.

```

20 | ✓ | public async Task<RecommendationStateDto?> GetRecommendationStatus(string correlationId)
21 | | {
22 | |     return await _redis.GetAsync<RecommendationStateDto>(RedisKeyPrefix + correlationId);
23 | | }

```

Refleksion

I bagklogskabens lys er der nogle valg, jeg har truffet for Recipe Recommendation System, som jeg havde gjort anderledes, hvis jeg havde muligheden for at lave projektet forfra.

Recipe Recommendation System bruger et CQRS-lignende mønster, hvor read-model og write-model er separeret i forskellige databaser, men i systemets filstruktur er der ikke lavet en opsplitning af commands og queries, som ellers er med til at give overskuelighed og signalerer, at CQRS mønstret bliver brugt.

Jeg havde truffet et andet valg end at implementere asynkront request-reply mønstret med polling og Redis, fordi ved polling kan klienten blive ved med at polle efter resultatet, før forespørgsel er færdig og derved skabe overhead i form af øget trafik og belastning på systemet, som kan blive en flaskehals og føre til skaleringsproblemer pga. det unødige forbrug af ressourcer.[12] Brugen af Redis skaber også mere kompleks infrastruktur og afhængighed af endnu en service, som øger risikoen for fejl.

På grund af tidsbegrænsning og minimumskrav for rapportens størrelse har der været mange emner, jeg har været nødt til at scope ud. Jeg kunne godt have tænkt mig at lave benchmarking af forskellige databaser for at give endnu bedre argumenter for, at de databaser jeg har valgt er de rigtige til de forskellige opgaver. Derudover har jeg ikke haft tid til at implementere en API Gateway, Authentication/Authorization og fejltolerance med circuit breakers og rate limits, som ellers er vigtige emner, som er med til at sikre robustheden og sikkerheden i et stort system. Systemet som det er nu fungerer blot som proof-of-concept og mangler både at blive udviklet færdigt og også at blive testet for at sikre, at hver af de enkelte dele fungerer, som de skal.

Konklusion

Formålet med teknologistudiet var at analysere den bedste måde at designe et microservice-baseret system med fokus på skalerbarhed og robusthed, og gennem valg truffet inden for arkitektur, design og teknologi er det mundet ud i proof-of-concept systemet "Recipe Recommendation System", som genererer opskriftsanbefalinger til sine slutbrugere gennem grafbaseret datamodellering og domænespecifik forespørgselslogik.

"Recipe Recommendation System" opnår skalerbarhed gennem systemets egne uafhængigt skalerbare microservices, som hver er designet med ét ansvar og én database af den type, som bedst håndterer opgaven og samtidig understøtter skalerbarhed.

"Recipe Recommendation System" opnår robusthed ved at acceptere eventuel konsistens for til gengæld at opnå partition tolerance og tilgængelighed, og ved at kommunikationen mellem microservices er asynkron og via message broker, som understøtter replayability.

Litteraturliste

- [1] "The Polar Chart," *cs3240.org*. [Online]. Available: <https://www.cs3240.org/modules/process/the-polar-chart/>
- [2] "What is Kanban?" *Atlassian*. [Online]. Available: <https://www.atlassian.com/agile/kanban>
- [3] "Microservices vs. Monolith," *Atlassian*. [Online]. Available: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>
- [4] "CAP theorem," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/CAP_theorem
- [5] "Asynchronous Request-Reply pattern," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply>
- [6] IntelliSoft, "RabbitMQ vs Kafka: Choosing the Right Messaging System for Your Needs," *Medium*. [Online]. Available: <https://medium.com/@IntelliSoft/rabbitmq-vs-kafka-choosing-the-right-messaging-system-for-your-needs-85c7a6db38ad>
- [7] "In-Memory Cache," *Redis*. [Online]. Available: <https://redis.io/glossary/in-memory-cache/>
- [8] "Why Graph Databases?" *Neo4j*. [Online]. Available: <https://neo4j.com/why-graph-databases/>
- [9] "Native vs. Non-Native Graph Technology," *Neo4j Blog*. [Online]. Available: <https://neo4j.com/blog/cypher-and-gql/native-vs-non-native-graph-technology/>
- [10] N. Infonikhil, "SQL vs NoSQL: Which Database Is Best for Horizontal Scaling, Sharding and Replication," *Medium*. [Online]. Available: <https://medium.com/@infonikhil270/sql-vs-nosql-which-database-is-best-for-horizontal-scaling-sharding-and-replication-798947d8c1ac>
- [11] S. Brown, *The C4 Model for Software Development*. [Online]. Available: <https://c4model.com/>
- [12] "Event-Driven or Polling-Based," *Flyriver*. [Online]. Available: <https://www.flyriver.com/g/event-driven-or-polling-based>